UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Allan Di Pace Schmidt
Alysson Alcantara
Anderson de Souza Pereira
Guilherme Cardoso de Almeida
Luiz Felipe Amaral de Almeida
Mariana Luisa de Faria Santos
Whedja Silva Cavalcante

Leciona: Plataforma Cloud de Gestão Escolar e API

Cachoeira Paulista - SP / São Sebastião - SP
2026

UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Leciona: Plataforma Cloud de Gestão Escolar e API

Relatório Técnico-Científico apresentado na disciplina de Projeto Integrador para o curso de Bacharelado em Tecnologia da Informação da Universidade Virtual do Estado de São Paulo (UNIVESP).

Cachoeira Paulista - SP / São Sebastião - SP
2026

ALCANTARA, Alysson; ALMEIDA, Guilherme Cardoso de; ALMEIDA, Luiz Felipe Amaral de; CAVALCANTE, Whedja Silva; PEREIRA, Anderson de Souza; SANTOS, Mariana Luisa de Faria; SCHMIDT, Allan Di Pace. **Leciona: Plataforma Cloud de Gestão Escolar e API.** 31 f. Relatório Técnico-Científico. Bacharelado em Tecnologia da Informação – **Universidade Virtual do Estado de São Paulo**. Tutor: Laís de Ponte. Polo Cachoeira Paulista - SP / São Sebastião - SP, 2026.

RESUMO

A gestão operacional de escolas públicas municipais envolve tarefas administrativas complexas, entre as quais o controle de ausências e a alocação de professores substitutos se destaca pela urgência e pelo impacto direto na rotina dos alunos. Na Escola Municipal Patrícia Viviani, localizada no bairro Topolândia em São Sebastião (SP), esse controle ainda era realizado de forma manual, por meio de planilhas e comunicações informais, o que gerava atrasos, choques de horários e perda de informações. Diante desse cenário, o presente trabalho apresenta a evolução do sistema web "Leciona", desenvolvido com o framework Django e banco de dados PostgreSQL, cujo objetivo é centralizar a gestão de grades de horários, ausências, substituições e comunicados escolares em uma plataforma acessível pela nuvem. A metodologia adotada seguiu a abordagem de Design Thinking proposta pela UNIVESP, estruturada nas etapas de ouvir, criar e implementar, com participação ativa da equipe gestora da escola parceira. A arquitetura do sistema foi organizada em quatro módulos independentes (core, schedule, attendance e communication), seguindo princípios SOLID, com restrições de integridade implementadas diretamente no banco de dados. Como resultados parciais, o sistema conta com um painel administrativo baseado no tema Jazzmin, um dashboard analítico com gráficos dinâmicos via Chart.js, exportação de relatórios em PDF e ambiente totalmente conteinerizado com Docker. A solução inicial foi apresentada à comunidade escolar, que validou sua aplicabilidade e indicou melhorias para as próximas iterações do projeto.

PALAVRAS-CHAVE: Gestão escolar; Django; API RESTful; Computação em nuvem; Integração contínua.

LISTA DE FIGURAS

FIGURA 2.1 – Diagrama entidade-relacionamento do sistema Leciona **22**

FIGURA 2.2 – Dashboard analítico do sistema Leciona **23**

FIGURA 2.3 – Painel administrativo do Leciona com tema Jazzmin **24**

FIGURA 2.4 – Lista de professores cadastrados no painel administrativo **25**

FIGURA 2.5 – Lista de horários cadastrados com filtros **26**

LISTA DE TABELAS

TABELA 2.1 – Problemas identificados na gestão de ausências da escola parceira **20**

LISTA DE SIGLAS E ABREVIATURAS

ACID – Atomicidade, Consistência, Isolamento e Durabilidade

API – *Application Programming Interface*

ARIA – *Accessible Rich Internet Applications*

AVA – Ambiente Virtual de Aprendizagem

CI/CD – *Continuous Integration / Continuous Delivery*

HTML – *HyperText Markup Language*

HTTP – *HyperText Transfer Protocol*

JSON – *JavaScript Object Notation*

LGPD – Lei Geral de Proteção de Dados Pessoais

MTV – *Model-Template-View*

MVC – *Model-View-Controller*

NIST – *National Institute of Standards and Technology*

ORM – *Object-Relational Mapping*

REST – *Representational State Transfer*

SGBD – Sistema de Gerenciamento de Banco de Dados

SOLID – *Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*

SQL – *Structured Query Language*

SRP – *Single Responsibility Principle* (Princípio da Responsabilidade Única)

TCLE – Termo de Consentimento Livre e Esclarecido

UNIVESP – Universidade Virtual do Estado de São Paulo

WCAG – *Web Content Accessibility Guidelines*

WSGI – *Web Server Gateway Interface*

YAML – *YAML Ain't Markup Language*

SUMÁRIO

1 INTRODUÇÃO **8**

2 DESENVOLVIMENTO **10**

2.1 Objetivos 10

2.2 Justificativa e delimitação do problema 10

2.3 Fundamentação teórica 12

2.3.1 Gestão escolar e tecnologia da informação 12

2.3.2 Desenvolvimento web com o framework Django 13

2.3.3 Banco de dados relacional e integridade referencial 14

2.3.4 API RESTful e arquitetura orientada a serviços 14

2.3.5 Computação em nuvem e conteinerização 15

2.3.6 Integração contínua e testes automatizados 16

2.3.7 Acessibilidade web 17

2.4 Metodologia 17

2.4.1 Ouvir e interpretar o contexto 17

2.4.2 Criar e prototipar 18

2.4.3 Implementar e testar 19

2.5 Resultados preliminares: solução inicial 20

2.5.1 Resultados da etapa de ouvir 20

2.5.2 Resultados da etapa de criar 20

2.5.3 Resultados da etapa de implementar 22

2.6 Considerações parciais 27

REFERÊNCIAS **29**

ANEXOS **30**

APÊNDICES **31**

1 INTRODUÇÃO

A administração de uma escola pública envolve tarefas que vão muito além do pedagógico. Coordenações e secretarias lidam diariamente com matrículas, distribuição de turmas, controle de frequência docente e remanejamento de horários. Entre essas demandas, a gestão de ausências de professores se destaca pela imprevisibilidade: atestados médicos, licenças e afastamentos ocorrem sem aviso prévio e exigem resposta imediata para que turmas não fiquem ociosas. Lück (2009) aponta que a eficácia da gestão escolar depende da capacidade de articular processos administrativos e pedagógicos de forma integrada, o que se torna inviável quando ferramentas adequadas de organização não estão disponíveis.

Apesar do avanço da tecnologia em diversas áreas da educação, muitas escolas públicas municipais ainda dependem de planilhas avulsas, cadernos de anotação e comunicação por aplicativos de mensagem para resolver questões operacionais. Moran (2015) argumenta que a incorporação de tecnologias digitais na educação precisa ultrapassar o uso em sala de aula e alcançar também os processos de gestão, possibilitando decisões mais ágeis e fundamentadas em dados. Na prática, a ausência de sistemas informatizados para tarefas administrativas resulta em retrabalho, erros de comunicação e perda de informações relevantes.

Esse cenário se reproduz na Escola Municipal Patrícia Viviani, localizada no bairro Topolândia, em São Sebastião (SP). A escola funciona com múltiplas turmas e turnos, e o controle de substituições de professores é feito manualmente pela secretaria e pela vice-direção. Quando um professor comunica ausência, a equipe administrativa precisa verificar quais horários ficaram descobertos, identificar substitutos disponíveis e reorganizar a grade, tudo isso frequentemente no mesmo dia. Choques de horários, substituições duplicadas e a falta de registro centralizado são problemas recorrentes. A Vice-Diretora Marília Izabel confirmou que a escola não dispõe de nenhum sistema digital voltado a esse tipo de controle, o que torna o processo lento e sujeito a falhas.

No semestre anterior, durante o Projeto Integrador II, o grupo desenvolveu um protótipo do sistema Leciona para atender a essa demanda. Construído com o framework Django e banco de dados PostgreSQL, o protótipo permitia o cadastro de professores, disciplinas, turmas e horários, além de registrar ausências e sugerir substituições. No entanto, o sistema funcionava apenas em ambiente local, sem possibilidade de acesso remoto por parte da equipe escolar. Não havia testes automatizados que garantissem a integridade das regras de negócio (como a impossibilidade de alocar um mesmo professor em duas turmas no mesmo horário), e a interface, embora funcional, não oferecia painéis analíticos para apoiar a tomada de decisão da coordenação.

A continuidade do projeto no Projeto Integrador III se justifica tanto pela demanda real da escola parceira quanto pelo alinhamento com o tema norteador do semestre, que exige desenvolvimento de software web em nuvem com banco de dados, automação por script web

(JavaScript), acessibilidade, integração contínua, testes automatizados e fornecimento de API RESTful. Manter o projeto anterior, em vez de iniciar um novo, permite ao grupo aprofundar a solução: corrigir limitações identificadas na primeira versão, incorporar boas práticas de engenharia de software e entregar um produto com maior maturidade técnica. Sommerville (2011) destaca que a evolução de software é uma etapa natural e necessária do ciclo de vida de sistemas, na qual requisitos são refinados e a arquitetura é ajustada conforme o conhecimento adquirido durante o uso e o desenvolvimento.

A decisão de evoluir o Leciona também considerou o retorno da escola. Após a apresentação do protótipo no PI2, a Vice-Diretora indicou funcionalidades prioritárias, como a visualização consolidada de faltas por período e a comunicação antecipada de substituições aos professores. Essas demandas reforçaram a necessidade de uma API que permitisse o consumo dos dados por diferentes interfaces, além do painel web já existente.

Este relatório parcial apresenta a evolução do sistema Leciona no contexto do Projeto Integrador III. O foco recai sobre os módulos de gestão de horários, registro de ausências, alocação de substituições e comunicação com a equipe escolar. São descritas a arquitetura modular adotada, a estratégia de conteinerização com Docker para viabilizar o deploy em nuvem, a implementação de testes automatizados para validação de regras de negócio, a construção dos primeiros endpoints de API RESTful e o desenvolvimento de um dashboard analítico para suporte à gestão. O documento também detalha a metodologia empregada e os resultados parciais obtidos até o momento. O cronograma de atividades é apresentado no Plano de Ação (Apêndice B).

2 DESENVOLVIMENTO

2.1 OBJETIVOS

O objetivo geral deste projeto é evoluir o sistema Leciona, desenvolvido inicialmente no Projeto Integrador II, para uma plataforma de gestão escolar operacional com deploy em nuvem, capaz de garantir a integridade da alocação de horários por meio de restrições a nível de banco de dados e testes automatizados, além de disponibilizar uma API RESTful para consumo descentralizado dos dados de horários, ausências e substituições.

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. Refatorar a arquitetura do sistema em módulos independentes (*core*, *schedule*, *attendance*, *communication*), aplicando princípios SOLID de responsabilidade única e separação de interesses.
2. Implementar restrições de integridade a nível de banco de dados (UniqueConstraint, ForeignKey, validações customizadas) para impedir choques de horários entre professores e entre turmas.
3. Desenvolver um dashboard analítico com gráficos dinâmicos (Chart.js 4.x) para visualização de indicadores de ausências, substituições e distribuição de turmas por período.
4. Conteinerizar a aplicação com Docker e Docker Compose, garantindo paridade entre os ambientes de desenvolvimento, teste e produção.
5. Criar endpoints de API RESTful (JSON) para consumo assíncrono de dados de horários e faltas por interfaces externas.
6. Implementar pipeline de Integração Contínua via GitHub Actions para execução automática de testes a cada *push*.
7. Desenvolver funcionalidade de exportação de relatórios em PDF (ReportLab), com cabeçalho institucional, tabela de dados e rodapé com data de emissão.
8. Validar a solução junto à comunidade escolar da Escola Municipal Patrícia Viviani, coletando *feedback* para iterações futuras.

2.2 JUSTIFICATIVA E DELIMITAÇÃO DO PROBLEMA

A pergunta norteadora desta pesquisa pode ser formulada da seguinte maneira: como garantir a alta disponibilidade, a integridade de dados e a facilidade de integração em um sistema de alocação de substituições emergenciais de professores em escolas públicas municipais?

A Escola Municipal Patrícia Viviani, localizada no bairro Topolândia, em São Sebastião (SP), atende alunos do Ensino Fundamental em múltiplas turmas distribuídas nos turnos matutino, vespertino e integral. A equipe administrativa da escola, composta pela Vice-Diretora Marília Izabel e por funcionários de secretaria, é responsável por organizar diariamente a grade de aulas. Quando um professor comunica ausência (por atestado médico, licença, luto ou outro motivo), essa equipe precisa, no mesmo dia, verificar os horários descobertos, localizar substitutos disponíveis e redistribuir a grade. Todo esse processo é feito manualmente, com anotações em cadernos e planilhas impressas, e a comunicação das substituições ocorre por meio de mensagens informais em aplicativos de celular, sem garantia de que todos os envolvidos recebam a informação a tempo.

As consequências desse fluxo manual são concretas. Turmas ficam sem professor quando a substituição não é alocada com rapidez suficiente. Choques de horários acontecem porque não há mecanismo que impeça a alocação de um mesmo substituto em duas turmas simultaneamente. A secretaria dedica parte significativa do expediente a tarefas que poderiam ser automatizadas, desviando esforço de outras demandas administrativas. Além disso, a ausência de dados consolidados impede que a coordenação identifique padrões recorrentes de falta (por exemplo, concentrações em determinados dias da semana ou períodos do ano) que poderiam subsidiar ações preventivas.

Do ponto de vista acadêmico, o projeto oferece ao grupo a oportunidade de aplicar, em um cenário real, conceitos estudados ao longo do curso de Tecnologia da Informação: modelagem de banco de dados relacional com restrições de integridade (ELMASRI; NAVATHE, 2011), desenvolvimento web seguindo o padrão arquitetural MTV do Django (DJANGO SOFTWARE FOUNDATION, 2025), conteinerização de aplicações com Docker (DOCKER INC., 2024), construção de API RESTful (FIELDING, 2000) e práticas de integração contínua (HUMBLE; FARLEY, 2010). A intersecção entre engenharia de software e gestão escolar permite que o grupo vivencie o ciclo completo de desenvolvimento, desde a elicitação de requisitos junto ao usuário final até o deploy e a coleta de *feedback*.

A contribuição prática do projeto se materializa em um sistema gratuito e acessível via navegador, que centraliza o registro de ausências, automatiza a verificação de conflitos de horários, gera relatórios em PDF e apresenta indicadores visuais em um painel analítico. Por ser conteinerizado, o sistema pode ser implantado em qualquer provedor de nuvem sem ajustes na configuração. A escola parceira passa a contar com uma ferramenta que reduz o retrabalho da secretaria e fornece dados para a gestão.

O projeto dá continuidade ao trabalho realizado no Projeto Integrador II, quando o grupo construiu o protótipo inicial do Leciona. Naquela etapa, a Vice-Diretora Marília Izabel validou a proposta e indicou melhorias prioritárias, como a necessidade de acesso remoto (o protótipo funcionava apenas localmente), a visualização consolidada de faltas por período e a comunicação estruturada de substituições. Essas demandas orientaram o escopo do PI3, que concentra esforços

na conteinerização, na API e no dashboard analítico.

Cabe registrar que o sistema lida com dados pessoais de professores (nome e telefone). A coleta dessas informações foi autorizada por meio de Termo de Consentimento Livre e Esclarecido (TCLE) aplicado junto aos participantes. O banco de dados PostgreSQL oferece mecanismos de controle de acesso e criptografia de conexão que contribuem para a proteção desses dados, em conformidade com os princípios da Lei Geral de Proteção de Dados Pessoais (LGPD).

2.3 FUNDAMENTAÇÃO TEÓRICA

2.3.1 Gestão escolar e tecnologia da informação

A gestão escolar, segundo Lück (2009), não se restringe à administração de recursos financeiros e materiais. Ela envolve a articulação de processos pedagógicos, administrativos e comunitários, de modo que as decisões tomadas pela equipe gestora repercutam diretamente na qualidade do ensino oferecido aos alunos. A autora enfatiza que uma gestão eficaz depende da capacidade de integrar essas dimensões, o que exige instrumentos de organização que vão além do caderno de anotações e da planilha impressa.

Nas escolas públicas municipais brasileiras, as dificuldades administrativas são agravadas pela escassez de recursos humanos e tecnológicos. Coordenadores e secretários acumulam funções que incluem controle de frequência docente, organização de grades horárias, registro de matrículas e comunicação com pais e professores. Quando essas tarefas dependem exclusivamente de processos manuais, o risco de erros aumenta e o tempo disponível para atividades de planejamento pedagógico diminui. A falta de registros digitais centralizados impede, ainda, a construção de indicadores que poderiam orientar decisões da coordenação.

Moran (2015) argumenta que a incorporação de tecnologias digitais na educação precisa ir além da sala de aula e alcançar os processos de gestão institucional. O autor defende que sistemas informatizados aplicados à administração escolar podem acelerar rotinas operacionais, reduzir a incidência de falhas humanas e gerar dados que subsidiem a tomada de decisão. A digitalização dos processos de gestão, contudo, precisa considerar a realidade local da escola: infraestrutura de rede, familiaridade da equipe com ferramentas digitais e a necessidade de interfaces acessíveis que não imponham uma curva de aprendizado íngreme.

> A gestão escolar constitui uma dimensão e um enfoque de atuação em educação, que objetiva promover a organização, a mobilização e a articulação de todas as condições materiais e humanas necessárias para garantir o avanço dos processos socioeducacionais dos estabelecimentos de ensino, orientados para a promoção efetiva da aprendizagem dos alunos (LÜCK, 2009, p. 24).

Essa definição reforça que a gestão escolar não é um fim em si, mas um meio para

viabilizar o processo educacional. Nesse enquadramento, um sistema como o Leciona atua como instrumento de apoio, automatizando tarefas repetitivas (registro de faltas, verificação de disponibilidade de substitutos, geração de relatórios) para que a equipe gestora possa concentrar seus esforços nas decisões pedagógicas.

2.3.2 Desenvolvimento web com o framework Django

O Django é um *framework* web de alto nível, escrito em Python, que segue o padrão arquitetural MTV (Model-Template-View). Nesse padrão, o Model define a estrutura dos dados e as regras de validação, o Template é responsável pela camada de apresentação (HTML renderizado), e a View atua como intermediária, recebendo requisições HTTP, consultando os modelos e retornando respostas ao cliente. Embora a nomenclatura difira do padrão MVC (Model-View-Controller) amplamente documentado na literatura de engenharia de software, a correspondência funcional é direta: o Template do Django equivale à View do MVC, e a View do Django equivale ao Controller (DJANGO SOFTWARE FOUNDATION, 2025).

O ciclo de uma requisição no Django segue um fluxo bem definido. O servidor recebe a requisição HTTP e a encaminha ao sistema de roteamento (URLconf), que mapeia a URL para uma View específica. A View processa a lógica de negócio, interage com o banco de dados por meio do ORM (Object-Relational Mapping) e passa os dados para o Template, que gera o HTML de resposta. Esse ORM permite que o desenvolvedor manipule registros do banco de dados usando objetos Python, sem a necessidade de escrever SQL manualmente na maioria das operações. Consultas mais elaboradas, como agregações com `ExtractMonth` e `ExtractIsoWeekDay` (utilizadas no dashboard do Leciona), também são suportadas pela API de consultas do Django.

A modularização do sistema em aplicações Django independentes segue os princípios SOLID descritos por Martin (2009). O Princípio da Responsabilidade Única (SRP) orienta que cada módulo deve ter um, e apenas um, motivo para mudar. No Leciona, o módulo *core* concentra as entidades fundamentais (Teacher, Subject, ClassGroup, SchoolSegment, SchoolPeriod, Justification), o módulo *schedule* cuida exclusivamente da alocação de horários, o módulo *attendance* gerencia ausências e substituições, e o módulo *communication* trata do envio de comunicados. Essa separação permite que alterações em um módulo (por exemplo, a adição de um novo tipo de comunicado) não afetem o funcionamento dos demais.

A Django Software Foundation mantém documentação oficial extensa e atualizada, cobrindo desde a instalação até funcionalidades avançadas como *middleware*, sinais, *managers* customizados e o sistema de administração automática. Essa documentação, associada a uma comunidade ativa e a um ciclo de lançamento previsível, torna o Django uma escolha adequada para projetos acadêmicos que precisam de uma base madura e bem documentada (DJANGO SOFTWARE FOUNDATION, 2025).

2.3.3 Banco de dados relacional e integridade referencial

O modelo relacional, formalizado por Codd (1970) e amplamente detalhado por Elmasri e Navathe (2011), organiza os dados em tabelas (relações) compostas por linhas (tuplas) e colunas (atributos). As relações entre tabelas são estabelecidas por meio de chaves primárias e chaves estrangeiras, e a normalização do esquema reduz a redundância dos dados e as anomalias de atualização. A adoção de um SGBD relacional confere ao sistema um conjunto de garantias (as propriedades ACID: atomicidade, consistência, isolamento e durabilidade) que são fundamentais para aplicações que manipulam dados sensíveis, como registros de frequência docente.

O PostgreSQL, utilizado no Leciona, é um SGBD relacional de código aberto reconhecido pela robustez, pela conformidade com o padrão SQL e pelo suporte a funcionalidades avançadas como tipos de dados customizados, indexação parcial e extensões. Sua escolha neste projeto se justifica tanto pela maturidade do software quanto pela integração nativa com o ORM do Django, que gera automaticamente as migrações de esquema a partir dos modelos Python.

Um dos mecanismos centrais para garantir a integridade dos dados no Leciona é o uso de *constraints* (restrições) definidas diretamente no banco. O modelo Schedule, por exemplo, possui duas restrições de unicidade (`UniqueConstraint`): a primeira impede que um professor seja alocado em duas turmas diferentes no mesmo dia e horário; a segunda impede que uma turma receba duas aulas simultâneas. Essas restrições são aplicadas pelo PostgreSQL no momento da inserção ou atualização dos dados, independentemente da camada de aplicação. Isso significa que, mesmo que uma validação na View ou no formulário falhe por algum motivo, o banco de dados impede a criação de registros inconsistentes (ELMASRI; NAVATHE, 2011).

Além das restrições de unicidade, o Leciona utiliza chaves estrangeiras (`ForeignKey`) para garantir a integridade referencial entre as tabelas. Uma ausência (`TeacherAbsence`) sempre referencia um professor e uma justificativa existentes; um horário (`Schedule`) sempre referencia uma turma, um professor e uma disciplina válidos. A indexação dos campos de data (`db_index=True` em `start_date` e `end_date`) acelera as consultas de filtragem por período, que são frequentes nas operações de geração de relatórios e de alimentação dos gráficos do dashboard.

2.3.4 API RESTful e arquitetura orientada a serviços

O estilo arquitetural REST (Representational State Transfer), proposto por Fielding (2000) em sua tese de doutorado, define um conjunto de restrições para a construção de sistemas distribuídos na Web. Entre os princípios fundamentais estão a interface uniforme (uso padronizado dos métodos HTTP), a comunicação *stateless* (cada requisição contém todas as informações necessárias para ser processada), a separação entre cliente e servidor, e a possibilidade de cache das respostas.

A adoção de uma API RESTful traz vantagens concretas para o Leciona. Ao expor os dados em formato JSON por meio de endpoints bem definidos, o sistema desacopla a camada de apresentação da lógica de negócio. O endpoint `/chartJSON/`, por exemplo, retorna os dados de ausências e substituições dos últimos seis meses em formato JSON, que são consumidos de forma assíncrona pelo Chart.js no navegador do usuário. Essa separação permite que, futuramente, aplicações móveis ou painéis externos consumam os mesmos dados sem alterações no *backend*.

A interoperabilidade proporcionada pela API facilita a integração com outros sistemas eventualmente utilizados pela secretaria de educação do município. Pressman e Maxim (2016) destacam que a construção de interfaces bem definidas entre componentes de software é um dos pilares da engenharia de software moderna, pois reduz o acoplamento entre módulos e permite a evolução independente de cada parte do sistema. No contexto do Leciona, a API representa o primeiro passo para transformar o sistema de um painel web monolítico em uma plataforma orientada a serviços.

2.3.5 Computação em nuvem e conteinerização

O National Institute of Standards and Technology (NIST) define computação em nuvem como um modelo que permite o acesso ubíquo, conveniente e sob demanda a um conjunto compartilhado de recursos computacionais configuráveis (redes, servidores, armazenamento, aplicações e serviços) que podem ser provisionados e liberados rapidamente, com esforço mínimo de gerenciamento ou interação com o provedor de serviços (MELL; GRANCE, 2011). Essa definição abrange os modelos de serviço IaaS, PaaS e SaaS, e o Leciona se posiciona como uma aplicação que pode ser implantada em qualquer um desses modelos, graças à conteinerização.

O Docker é uma plataforma de conteinerização que empacota a aplicação e todas as suas dependências (sistema operacional base, bibliotecas, *runtime* Python, pacotes) em uma imagem portátil. O Dockerfile do Leciona utiliza a imagem base `python:3.10-slim`, instala as dependências de sistema necessárias para a compilação do `psycopg2` (adaptador PostgreSQL para Python) e configura o Gunicorn como servidor WSGI com três *workers* na porta 8080. Essa imagem pode ser executada em qualquer máquina que tenha o Docker instalado, eliminando o problema clássico de divergência entre ambientes de desenvolvimento e produção (DOCKER INC., 2024).

O Docker Compose, por sua vez, permite a definição de aplicações compostas por múltiplos contêineres em um único arquivo YAML (`docker-compose.yml`). No Leciona, o Compose orquestra dois serviços: `db` (PostgreSQL 15 Alpine, exposto na porta 5433) e `web` (aplicação Django, exposta na porta 8080). O volume `postgres_data` garante a persistência dos dados do banco entre reinicializações dos contêineres. Com o comando `docker-compose up -d -build`, todo o ambiente é construído e iniciado automaticamente, o que simplifica tanto o desenvolvimento local quanto a implantação em servidores remotos.

Humble e Farley (2010) argumentam que a paridade entre ambientes é um pré-requisito para entregas confiáveis de software. Quando o ambiente de desenvolvimento difere do ambiente de produção em versões de bibliotecas, configurações de sistema operacional ou versões do banco de dados, erros que não se manifestam localmente podem surgir em produção. A conteinerização com Docker resolve esse problema ao garantir que o mesmo artefato (a imagem) é executado em todos os estágios do pipeline.

2.3.6 Integração contínua e testes automatizados

A Integração Contínua (CI), conforme definida por Humble e Farley (2010), é a prática de integrar o código de todos os desenvolvedores em um repositório compartilhado diversas vezes ao dia, executando uma *build* automatizada (que inclui compilação, testes e verificações estáticas) a cada integração. O objetivo é detectar erros o mais cedo possível no ciclo de desenvolvimento, reduzindo o custo de correção e aumentando a confiança da equipe na base de código.

O GitHub Actions é a plataforma de CI/CD integrada ao GitHub que permite definir *workflows* em arquivos YAML no diretório `.github/workflows/`. Cada *workflow* é composto por *jobs* que executam uma sequência de *steps* em máquinas virtuais provisionadas sob demanda. Para o Leciona, o *workflow* de CI é acionado a cada *push* e a cada abertura de *pull request*, executando a suíte de testes do Django em um ambiente com PostgreSQL configurado como serviço.

Os testes automatizados são a base da CI. Sommerville (2011) classifica os testes em unitários (que verificam o comportamento de componentes isolados), de integração (que verificam a interação entre componentes) e de sistema (que verificam o comportamento do sistema como um todo). No Leciona, os testes unitários validam regras de negócio críticas: a impossibilidade de um professor substituir a si mesmo (`TeacherSubstitution.clean()`), a obrigatoriedade de que a data de início preceda a data de término (`TeacherAbsence.clean()`), e a verificação de que o professor está habilitado a lecionar a disciplina atribuída (`Schedule.clean()`). As restrições de unicidade no banco de dados complementam esses testes, formando uma rede de proteção em duas camadas (aplicação e banco).

> Testing can only show the presence of errors, not their absence. Nevertheless, the value of software testing lies precisely in the systematic detection of faults that would otherwise reach production and affect users (SOMMERVILLE, 2011, p. 208).

Essa observação, que remonta ao trabalho de Dijkstra (1972), reforça a importância de combinar testes automatizados com outras estratégias de verificação, como revisão de código e *constraints* de banco de dados, para maximizar a cobertura de detecção de erros.

2.3.7 Acessibilidade web

As Diretrizes de Acessibilidade para Conteúdo Web (WCAG) 2.1, publicadas pelo World Wide Web Consortium, estabelecem critérios técnicos para que conteúdos e interfaces web sejam acessíveis a pessoas com diferentes tipos de deficiência (W3C, 2018). As diretrizes estão organizadas em quatro princípios (perceptível, operável, compreensível e robusto) e três níveis de conformidade (A, AA e AAA). O nível AA é considerado referência para a maioria das aplicações web governamentais e educacionais.

O Leciona utiliza o Jazzmin como tema para o painel administrativo do Django. O Jazzmin é construído sobre o AdminLTE 3, que por sua vez é baseado no Bootstrap 4. Essa cadeia de dependências confere ao sistema uma interface responsiva, que se adapta a diferentes tamanhos de tela (desktops, tablets e smartphones), além de oferecer componentes com atributos ARIA (*Accessible Rich Internet Applications*) que facilitam a interação por meio de leitores de tela. Os formulários do painel administrativo utilizam *labels* associados a campos de entrada, contrastes de cor adequados e navegação por teclado.

A escolha de um tema baseado em Bootstrap também contribui para a consistência visual da interface. Elementos como botões, tabelas, cards e menus seguem um padrão visual uniforme, reduzindo a carga cognitiva do usuário. Para a equipe administrativa da escola, que pode não ter familiaridade com sistemas complexos, essa consistência facilita o aprendizado e a adoção da ferramenta. Moran (2015) destaca que a tecnologia aplicada à educação só cumpre seu papel quando é efetivamente utilizada pelos atores envolvidos, o que depende, em grande medida, da usabilidade e da acessibilidade da interface oferecida.

2.4 METODOLOGIA

A metodologia adotada segue o Design Thinking proposto pela UNIVESP como framework para os Projetos Integradores, estruturado em três etapas: ouvir, criar e implementar. A seguir, descrevem-se as estratégias empregadas em cada etapa, os instrumentos de coleta de dados e as decisões técnicas tomadas durante o desenvolvimento.

2.4.1 Ouvir e interpretar o contexto

O projeto foi realizado em parceria com a Escola Municipal Patrícia Viviani, localizada no bairro Topolândia, em São Sebastião (SP). A escola atende turmas do Ensino Fundamental distribuídas nos turnos matutino, vespertino e integral, totalizando um corpo docente que ultrapassa trinta professores. A principal interlocutora do grupo na escola foi a Vice-Diretora Marília Izabel, que participou de todas as etapas de validação.

A coleta de informações ocorreu por meio de reuniões presenciais e remotas com a equipe

administrativa da escola. Na primeira reunião, o grupo apresentou o protótipo desenvolvido no PI2 e solicitou que a Vice-Diretora avaliasse as funcionalidades existentes e indicasse lacunas. Antes da coleta formal de dados, foi aplicado o Termo de Consentimento Livre e Esclarecido (TCLE) a todos os participantes, garantindo o uso ético das informações fornecidas.

Os problemas levantados durante essa fase foram: (a) o registro manual de ausências de professores, que dependia de cadernos e planilhas sem padronização; (b) choques de horários causados pela alocação manual de substitutos, sem verificação de disponibilidade; (c) comunicação descentralizada, em que avisos de substituição eram transmitidos por mensagens informais, sem garantia de recebimento por todos os envolvidos; (d) ausência de dados consolidados para identificar padrões de falta por período, dia da semana ou justificativa. A Vice-Diretora também apontou que o protótipo do PI2, por funcionar apenas localmente, não era utilizável no cotidiano da escola.

A partir dessas informações, o grupo definiu as prioridades do PI3: conteinerizar a aplicação para permitir acesso remoto, implementar restrições de integridade no banco de dados para eliminar choques de horários, construir um dashboard com indicadores visuais e estruturar a comunicação de substituições por meio de um módulo dedicado.

2.4.2 Criar e prototipar

Com os requisitos refinados, o grupo tomou a decisão de refatorar a base de código existente, em vez de iniciar um novo projeto. Essa escolha se baseou em dois fatores: o aproveitamento das entidades e regras de negócio já modeladas no PI2, e a possibilidade de demonstrar a evolução de um sistema ao longo de seu ciclo de vida, conforme preconizado por Sommerville (2011).

A primeira decisão técnica foi a reorganização do código em quatro módulos Django com responsabilidades bem definidas. O módulo *core* concentra as entidades fundamentais (Teacher, Subject, ClassGroup, SchoolSegment, SchoolPeriod, Justification). O módulo *schedule* trata exclusivamente da alocação de horários, com o modelo Schedule e suas restrições de unicidade. O módulo *attendance* gerencia o registro de ausências (TeacherAbsence) e substituições (TeacherSubstitution), incluindo validações que impedem a auto-substituição e verificam a consistência de datas. O módulo *communication* cuida do envio de comunicados tipificados (comunicação, evento, observação, pedagógico), com possibilidade de vinculação a turmas e professores específicos.

A modelagem do banco de dados foi revisada para incorporar restrições de integridade diretamente no PostgreSQL. Além das chaves estrangeiras e das validações no método `clean()` dos modelos Django, foram adicionadas duas `UniqueConstraint` ao modelo Schedule, que impedem, respectivamente, a duplicação de um professor em duas turmas no mesmo horário e

a duplicação de aulas em uma mesma turma no mesmo horário. Essas restrições atuam como camada de proteção adicional, independente da aplicação.

Para o dashboard, o grupo prototipou três gráficos utilizando Chart.js 4.x: um gráfico de linhas comparando ausências e substituições nos últimos seis meses, um gráfico de barras mostrando a distribuição de ausências por dia da semana, e um gráfico *doughnut* com a distribuição de turmas por período. Os dados são obtidos por meio de consultas agregadas no banco (usando `ExtractMonth` e `ExtractIsoWeekDay` do ORM do Django) e entregues ao *frontend* via JSON.

A conteinerização foi definida com Docker Compose, orquestrando dois serviços: o banco de dados PostgreSQL 15 Alpine e a aplicação Django servida pelo Gunicorn. A imagem Docker utiliza a base `python:3.10-slim` e instala apenas as dependências necessárias para compilação do `psycopg2` e da biblioteca Pillow, resultando em uma imagem leve e reprodutível.

Para facilitar o teste da interface e do dashboard, o grupo criou um comando de *seed* (`python manage.py seed_db`) que utiliza a biblioteca Faker com locale `pt_BR` para gerar dados realistas: 40 professores com nomes e telefones brasileiros, 20 turmas distribuídas em quatro segmentos e quatro períodos, 11 disciplinas, 6 justificativas de ausência, horários de aula com tratamento de conflitos, 50 ausências distribuídas nos últimos seis meses (com 70% de chance de substituição associada) e 20 comunicados.

2.4.3 Implementar e testar

A implementação seguiu a ordem definida na prototipação. A refatoração em módulos foi realizada primeiro, seguida pela adição das restrições de banco de dados, pela construção do dashboard, pela conteinerização e pela exportação de relatórios em PDF.

O deploy em Docker foi validado localmente com o comando `docker-compose up -d -build`, que constrói a imagem da aplicação, inicia o banco de dados PostgreSQL e expõe o sistema na porta 8080. O volume persistente `postgres_data` garante que os dados do banco sobrevivam a reinicializações dos contêineres. O seed de dados foi executado dentro do contêiner web para popular o banco com dados de teste.

Os testes unitários foram direcionados às regras de negócio mais críticas: a validação de que um professor não pode substituir a si mesmo, a validação de que datas de início devem preceder datas de término, a verificação de que um professor só pode lecionar disciplinas para as quais está habilitado, e a aplicação das restrições de unicidade que impedem choques de horários. Esses testes são executados pelo *test runner* do Django com banco de dados de teste em memória.

A apresentação do sistema à escola parceira foi planejada para a Quinzena 4 do calendário do PI3. O objetivo é coletar *feedback* qualitativo da Vice-Diretora e da secretaria sobre a usabilidade do painel administrativo, a clareza dos gráficos do dashboard e a utilidade da

funcionalidade de exportação em PDF. As sugestões recebidas serão priorizadas e incorporadas nas iterações seguintes, conforme a capacidade do grupo no período restante do semestre.

2.5 RESULTADOS PRELIMINARES: SOLUÇÃO INICIAL

Os resultados preliminares do projeto são apresentados a seguir, organizados conforme as três etapas da metodologia Design Thinking: ouvir, criar e implementar.

2.5.1 Resultados da etapa de ouvir

O contato com a Escola Municipal Patrícia Viviani permitiu identificar com precisão os problemas enfrentados pela equipe administrativa na gestão de ausências e substituições de professores. A Vice-Diretora Marília Izabel recebeu o protótipo do PI2 de forma positiva, destacando que a escola não possuía qualquer sistema digital voltado a esse tipo de controle. Ao mesmo tempo, indicou que a adoção efetiva dependeria de o sistema estar disponível em nuvem (acessível de qualquer computador da escola) e de oferecer garantias contra erros de alocação.

A Tabela 2.1 sintetiza os problemas identificados durante as reuniões com a escola.

Tabela 2.1: Problemas identificados na gestão de ausências da escola parceira

Problema	**Descrição**
Registro manual de ausências	Faltas de professores anotadas em cadernos ou planilhas, sem centralização digital
Choques de horários	Substituições alocadas manualmente, gerando sobreposição de professores em turmas diferentes no mesmo horário
Comunicação descentralizada	Avisos de substituição transmitidos por mensagens informais, sem garantia de que todos os envolvidos foram notificados
Ausência de histórico analítico	Sem dados consolidados para identificar padrões de ausência por período, dia da semana ou justificativa

Esses problemas confirmaram a pertinência da proposta e orientaram as prioridades de desenvolvimento para o PI3. A demanda por confiabilidade (evitar choques) foi traduzida em restrições de banco de dados; a demanda por acesso remoto foi traduzida em conteinerização e deploy em nuvem; a demanda por histórico analítico foi traduzida no dashboard com gráficos.

2.5.2 Resultados da etapa de criar

A refatoração do código resultou em uma arquitetura modular composta por quatro aplicações Django independentes, cada uma com seus modelos, views, templates e arquivos de

administração. A seguir, uma breve descrição de cada módulo:

- **core:** contém os modelos fundamentais do sistema. O modelo Teacher armazena nome, telefone (validado por expressão regular) e uma relação *many-to-many* com Subject, além de um campo booleano `is_substitute` que identifica professores substitutos. Os modelos ClassGroup, SchoolSegment e SchoolPeriod representam, respectivamente, turmas, segmentos escolares e períodos (manhã, tarde, noite, integral). O modelo Justification armazena os tipos de justificativa para ausências (médica, pessoal, maternidade, trânsito, luto, jurídica).

- **schedule:** contém o modelo Schedule, que relaciona turma, professor, disciplina, dia da semana e horário (início e fim). Duas `UniqueConstraint` garantem, a nível de banco de dados, que um professor não pode estar em duas turmas no mesmo horário e que uma turma não pode ter duas aulas simultâneas. O método `clean()` valida adicionalmente que o professor está habilitado a lecionar a disciplina atribuída.

- **attendance:** contém os modelos TeacherAbsence e TeacherSubstitution. O primeiro registra ausências com datas indexadas (`db_index=True`) e referência a professor e justificativa. O segundo registra substituições, com referências ao professor ausente e ao substituto. O método `clean()` de TeacherSubstitution impede que um professor substitua a si mesmo e valida a consistência das datas.

- **communication:** contém o modelo Message, que permite o registro de comunicados tipificados (comunicação, evento, observação, pedagógico), com data, descrição, imagem opcional e vinculação a turmas e professores por meio de relações *many-to-many*.

O modelo de dados resultante reflete as regras de negócio levantadas junto à escola. As chaves estrangeiras garantem a integridade referencial, as restrições de unicidade previnem conflitos de alocação, e as validações nos métodos `clean()` complementam essas restrições com verificações que o banco de dados relacional não consegue expressar sozinho (como a validação de habilitação do professor para a disciplina).

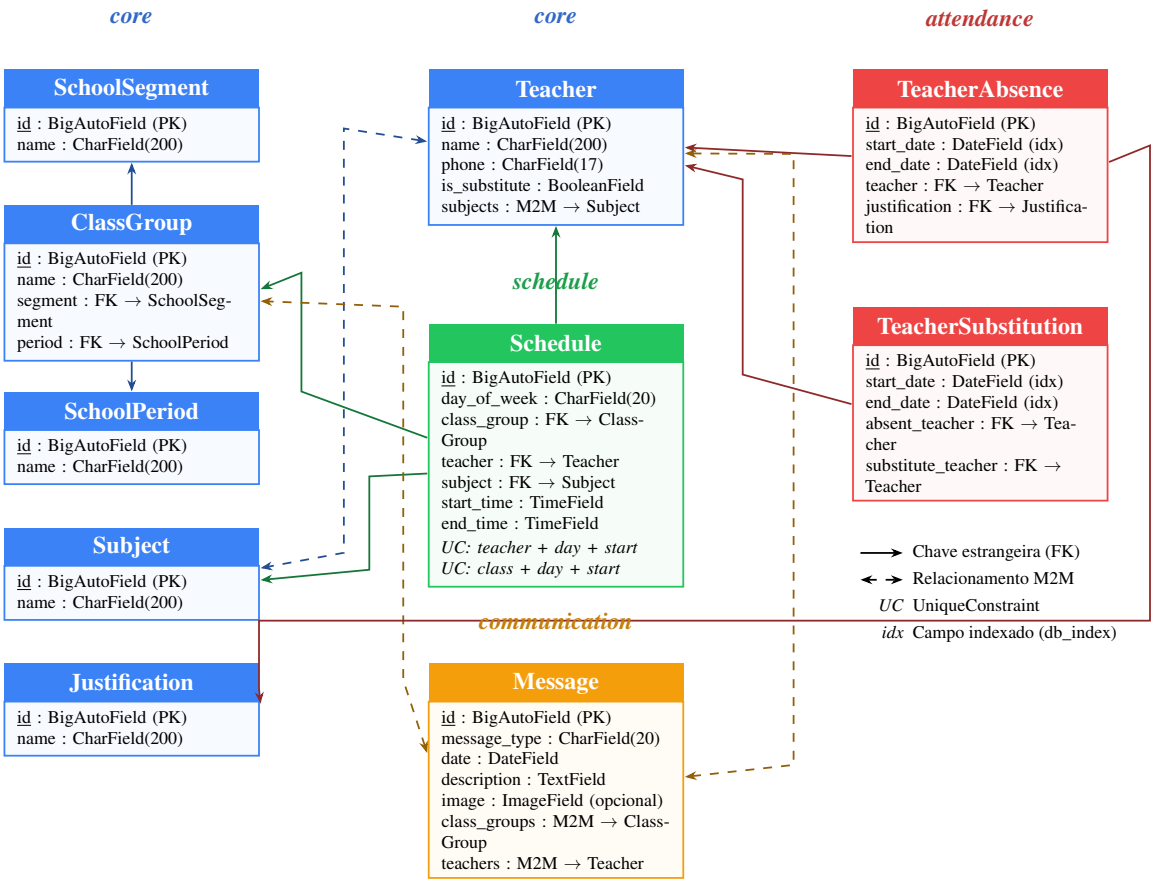

Figura 2.1: Diagrama entidade-relacionamento do sistema Leciona, organizado por módulo

A Figura 2.1 apresenta o diagrama entidade-relacionamento do sistema, organizado por módulo. As cores identificam a qual aplicação Django cada entidade pertence: azul para o módulo *core*, verde para *schedule*, vermelho para *attendance* e laranja para *communication*. As setas contínuas representam chaves estrangeiras e as setas tracejadas representam relacionamentos muitos-para-muitos.

2.5.3 Resultados da etapa de implementar

A implementação produziu um sistema funcional que cobre os requisitos definidos nas etapas anteriores. Os principais resultados são detalhados a seguir.

Dashboard analítico. A view `DashboardView`, acessível em `/statistics/`, apresenta três gráficos construídos com Chart.js 4.x e cards com contadores de professores, turmas e segmentos cadastrados. O primeiro gráfico é uma linha temporal que compara o número de ausências registradas com o número de substituições realizadas nos últimos seis meses. Os dados são obtidos por meio de consultas agregadas que utilizam `ExtractMonth` para agrupar os registros por mês, e são servidos como JSON pelo endpoint `/chartJSON/` (uma view baseada em `BaseLineChartView`). O segundo gráfico é um gráfico de barras que mostra a distribuição de ausências por dia da semana (segunda a sexta), utilizando `ExtractIsoWeekDay` para extrair o

dia da data de início de cada ausência. O terceiro gráfico é um *doughnut* que exibe a distribuição de turmas por período escolar (manhã, tarde, noite, integral), permitindo que a coordenação visualize rapidamente o equilíbrio da oferta entre turnos.

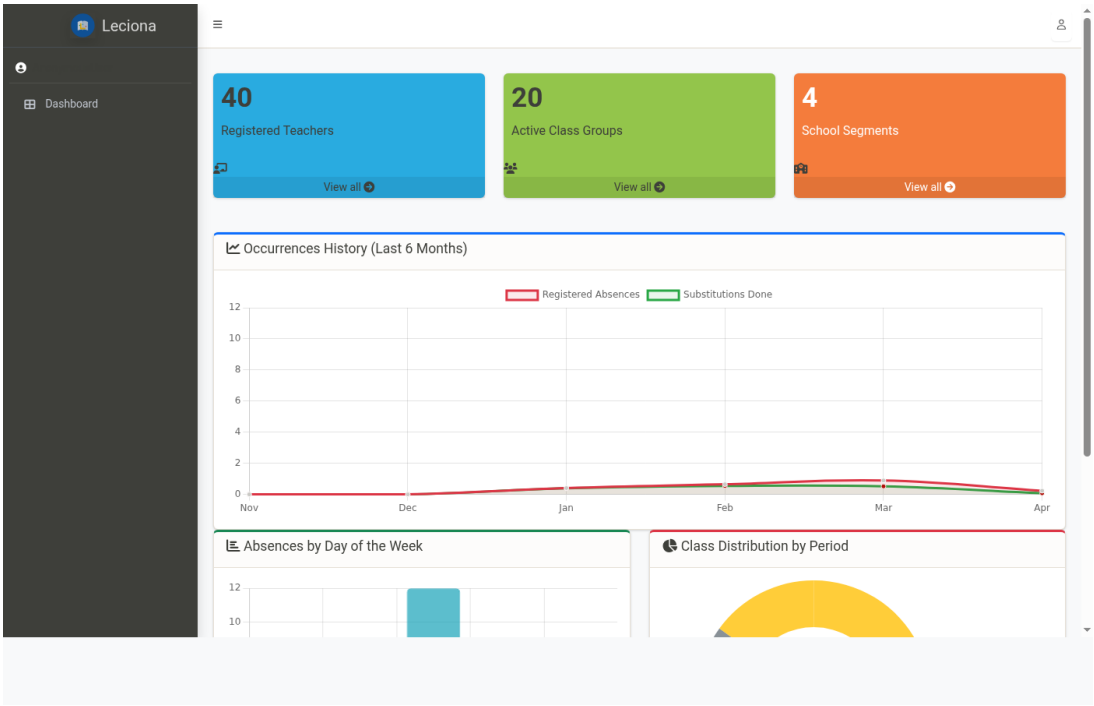

Figura 2.2: Dashboard analítico do sistema Leciona com gráficos de ausências e substituições

Painel administrativo. O painel administrativo do Django, personalizado com o tema Jazzmin (baseado em AdminLTE 3 e Bootstrap 4), oferece interfaces de cadastro, edição e consulta para todas as entidades do sistema: professores, disciplinas, turmas, segmentos, períodos, justificativas, horários, ausências, substituições e comunicados. Cada entidade possui uma classe `ModelAdmin` registrada, com configuração de campos exibidos em lista (`list_display`), filtros laterais (`list_filter`) e ações customizadas. A navegação pelo painel é intuitiva, com menus laterais organizados por módulo e suporte a busca rápida.

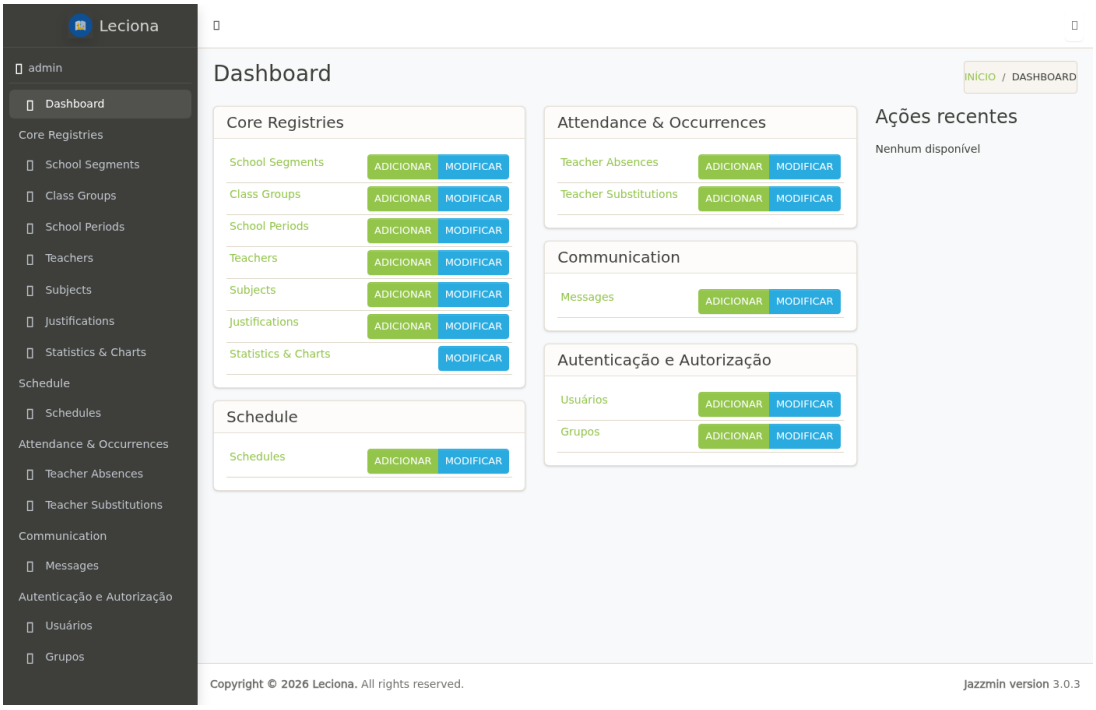

Figura 2.3: Painel administrativo do Leciona com tema Jazzmin

Exportação de relatórios em PDF. A funcionalidade de exportação em PDF foi implementada como uma ação customizada do Django Admin, disponível para todos os modelos registrados. A ação utiliza a biblioteca ReportLab para gerar documentos em formato A4 com cabeçalho institucional (logo do sistema e título), tabela com os dados selecionados, contagem total de registros e rodapé com data de emissão e nome do usuário que gerou o relatório. O administrador seleciona os registros desejados na lista, escolhe a ação “Export to PDF” e recebe o download do arquivo. Essa funcionalidade atende à demanda da secretaria por relatórios impressos de professores, turmas e ausências.

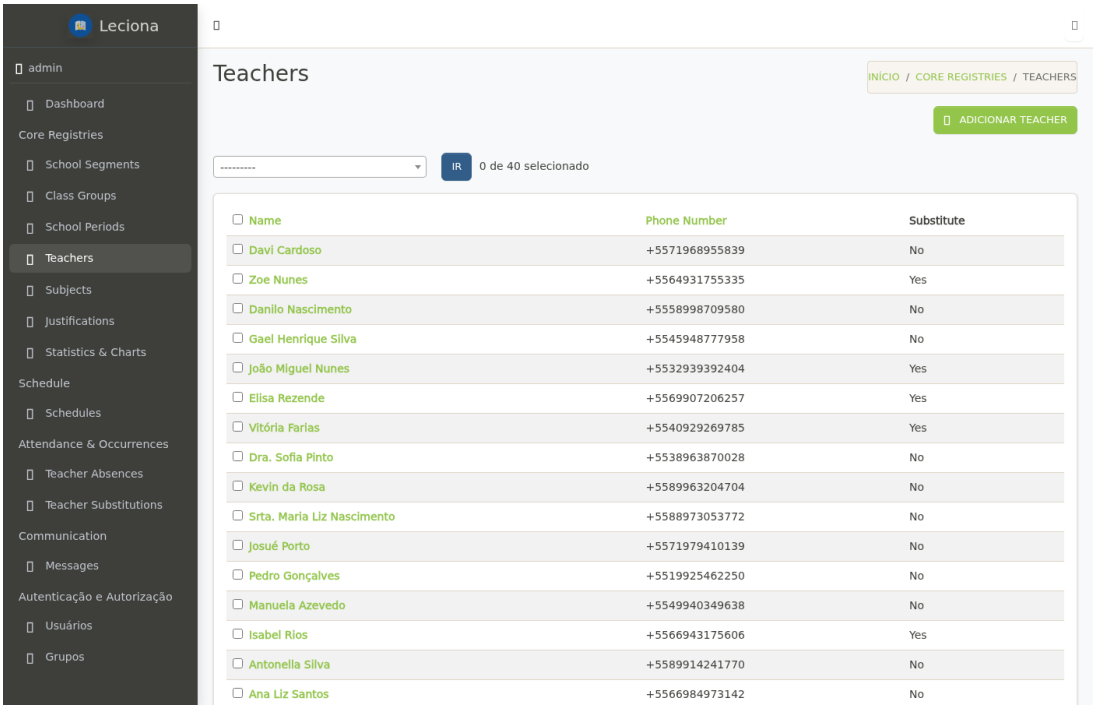

Figura 2.4: Lista de professores cadastrados no painel administrativo do Leciona

Gestão de horários com validação de conflitos. O cadastro de horários pelo painel administrativo passa por validação em duas camadas. Na primeira camada (aplicação), o método `clean()` do modelo Schedule verifica que o horário de início é anterior ao horário de término e que o professor está habilitado a lecionar a disciplina indicada. Na segunda camada (banco de dados), as `UniqueConstraint` impedem a inserção de registros que gerariam choque de professor ou choque de turma. Quando uma tentativa de inserção viola alguma dessas restrições, o Django exibe uma mensagem de erro descritiva no formulário, sem que o registro inválido seja persistido.

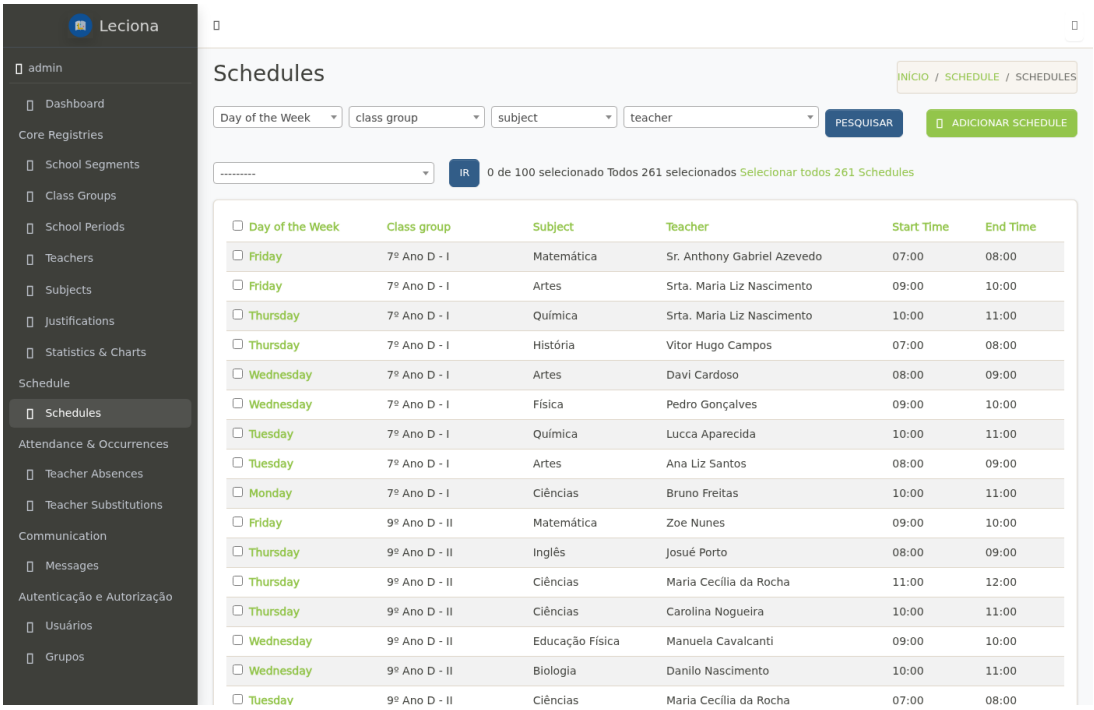

Figura 2.5: Lista de horários cadastrados com filtros por dia da semana, turma, disciplina e professor

Conteinerização com Docker. O ambiente completo do Leciona pode ser iniciado com o comando `docker-compose up -d -build`. O Docker Compose orquestra dois serviços: `db` (imagem `postgres:15-alpine`, porta 5433, volume persistente) e `web` (imagem construída a partir do Dockerfile do projeto, porta 8080). Em desenvolvimento, o comando do serviço `web` é sobrescrito para utilizar o servidor de desenvolvimento do Django (`runserver`); em produção, o Dockerfile define o Gunicorn com três *workers* como servidor padrão. O *seed* de dados é executado dentro do contêiner com `docker-compose exec web python manage.py seed_db`.

Seed de dados realistas. O comando `seed_db` gera um conjunto de dados compatível com o cenário real da escola parceira: 40 professores (25% marcados como substitutos) com nomes e telefones brasileiros gerados pelo Faker, 20 turmas vinculadas a 4 segmentos e 4 períodos, 11 disciplinas, 6 tipos de justificativa, horários de aula distribuídos de segunda a sexta em faixas de 07h às 12h (com tratamento automático de conflitos de `UniqueConstraint`), 50 ausências nos últimos 180 dias (70% com substituição associada) e 20 comunicados. Esse conjunto permite testar todos os gráficos do dashboard com dados volumétricos e variados, além de validar o comportamento das restrições de integridade sob carga.

Integração Contínua. O pipeline de CI foi configurado via GitHub Actions no arquivo `.github/workflows/ci.yml`. O *workflow* é acionado a cada *push* e a cada abertura de *pull request*, executando a suíte de testes do Django em um ambiente com PostgreSQL configurado como serviço. Essa automação garante que regressões sejam detectadas antes da integração ao ramo principal, reduzindo o risco de introdução de defeitos no código em produção.

Acessibilidade. A adoção do tema Jazzmin, construído sobre o AdminLTE 3 e o Bootstrap 4, confere ao painel administrativo uma interface responsiva com suporte a atributos ARIA, *labels* associados a campos de formulário, contrastes de cor adequados e navegação por teclado. Embora uma auditoria formal de conformidade com as WCAG 2.1 ainda não tenha sido realizada, as escolhas tecnológicas adotadas fornecem uma base sólida para acessibilidade, que será avaliada com maior rigor nas próximas iterações.

API RESTful. Até o momento, o endpoint `/chartJSON/` constitui o primeiro ponto de acesso à API, servindo dados de ausências e substituições em formato JSON para consumo assíncrono pelo Chart.js. A expansão da API com endpoints dedicados para consulta de horários, ausências e substituições por interfaces externas está planejada para as próximas quinzenas, dando continuidade ao objetivo de transformar o sistema em uma plataforma orientada a serviços.

Deploy em nuvem. A conteinerização com Docker e Docker Compose, já funcional, representa a etapa preparatória para o deploy em nuvem. O ambiente está configurado de forma que a mesma imagem utilizada localmente possa ser implantada em qualquer provedor de nuvem (AWS, GCP, Railway, entre outros) sem alterações na configuração. A efetivação do deploy remoto está prevista para as próximas iterações do projeto.

Em síntese, os resultados parciais demonstram que o sistema Leciona evoluiu de um protótipo local para uma aplicação conteinerizada, com restrições de integridade, dashboard analítico, exportação de relatórios, pipeline de integração contínua e o primeiro endpoint de API JSON. A validação junto à escola parceira, prevista para a Quinzena 4, fornecerá o *feedback* necessário para as últimas iterações do projeto dentro do semestre.

2.6 CONSIDERAÇÕES PARCIAIS

O desenvolvimento do sistema Leciona no contexto do Projeto Integrador III encontra-se em estágio avançado de execução. Dos oito objetivos específicos definidos, seis já foram total ou parcialmente alcançados: a refatoração em módulos independentes (objetivo 1), a implementação de restrições de integridade no banco de dados (objetivo 2), o dashboard analítico com gráficos dinâmicos (objetivo 3), a conteinerização com Docker (objetivo 4), a exportação de relatórios em PDF (objetivo 7) e a configuração do pipeline de integração contínua (objetivo 6).

Dois objetivos estão em andamento e serão concluídos nas próximas quinzenas: a expansão da API RESTful com endpoints dedicados para consumo externo de dados (objetivo 5) e a validação da solução junto à comunidade escolar (objetivo 8), prevista para a Quinzena 4. Adicionalmente, a efetivação do deploy em um provedor de nuvem complementará a conteinerização já realizada, permitindo que a escola parceira acesse o sistema remotamente.

As principais contribuições técnicas obtidas até o momento incluem a arquitetura modular com separação clara de responsabilidades, a proteção em duas camadas contra inconsistências de

dados (aplicação e banco), e a geração de indicadores visuais que subsidiam a tomada de decisão da equipe gestora. O *feedback* da escola parceira, a ser coletado na próxima etapa, orientará os ajustes finais e a priorização das funcionalidades remanescentes.

REFERÊNCIAS

CODD, E. F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, v. 13, n. 6, p. 377-387, 1970.

DIJKSTRA, E. W. The Humble Programmer. **Communications of the ACM**, v. 15, n. 10, p. 859-866, 1972.

DJANGO SOFTWARE FOUNDATION. **Django documentation.** Version 5.2. 2025. Disponível em: https://docs.djangoproject.com/en/5.2/. Acesso em: mar. 2026.

DOCKER INC. **Docker Documentation.** 2024. Disponível em: https://docs.docker.com/. Acesso em: mar. 2026.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados.** 6. ed. São Paulo: Pearson Addison Wesley, 2011.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures.** 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

HUMBLE, J.; FARLEY, D. **Continuous Delivery:** Reliable Software Releases through Build, Test, and Deployment Automation. Upper Saddle River: Addison-Wesley, 2010.

LÜCK, H. **Dimensões da gestão escolar e suas competências.** Curitiba: Positivo, 2009.

MARTIN, R. C. **Clean Code:** A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2009.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing.** Gaithersburg: National Institute of Standards and Technology, 2011. (Special Publication 800-145).

MORAN, J. M. Mudando a educação com metodologias ativas. In: SOUZA, C. A. de; MORALES, O. E. T. (Org.). **Convergências midiáticas, educação e cidadania:** aproximações jovens. Ponta Grossa: UEPG, 2015. p. 15-33.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software:** uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de Software.** 9. ed. São Paulo: Pearson Prentice Hall, 2011.

W3C – WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.1.** 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: mar. 2026.

ANEXOS

ANEXO A – Termo de Autorização da Empresa/Instituição

O Termo de Autorização da Empresa/Instituição, assinado pela responsável da Escola Municipal Patrícia Viviani, autoriza a realização do Projeto Integrador nas dependências da escola e o uso de dados coletados para fins acadêmicos. O documento original encontra-se arquivado junto ao grupo e uma cópia foi entregue à instituição parceira.

APÊNDICES

APÊNDICE A – Termo de Consentimento Livre e Esclarecido (TCLE)

O Termo de Consentimento Livre e Esclarecido foi aplicado aos participantes da pesquisa antes da coleta de dados junto à equipe administrativa da Escola Municipal Patrícia Viviani. O documento garante a participação voluntária, o direito de desistência a qualquer momento e a privacidade dos dados fornecidos. O modelo utilizado encontra-se arquivado junto ao grupo.

APÊNDICE B – Plano de Ação

O Plano de Ação detalha as atividades realizadas pelo grupo ao longo das sete quinzenas do Projeto Integrador III, incluindo responsáveis, datas de início e finalização, e observações sobre cada atividade. O documento foi entregue como artefato independente no Ambiente Virtual de Aprendizagem (AVA) da UNIVESP.